

Bloom as a project-learning coach: target architecture

How Bloom evolves from an AI-written checklist into a coach that helps each student make real things for their project, get feedback and improve them. It builds on the existing tables, actions and pages instead of replacing them.

Status proposal for review, nothing implemented Baseline Bloom as built (docs/bloom-as-built) Written 5 Oct 2026

MVP phase 1, needed to make Bloom interactive **PHASE 2** once the core works **LATER** advanced

00 Summary and key decisions

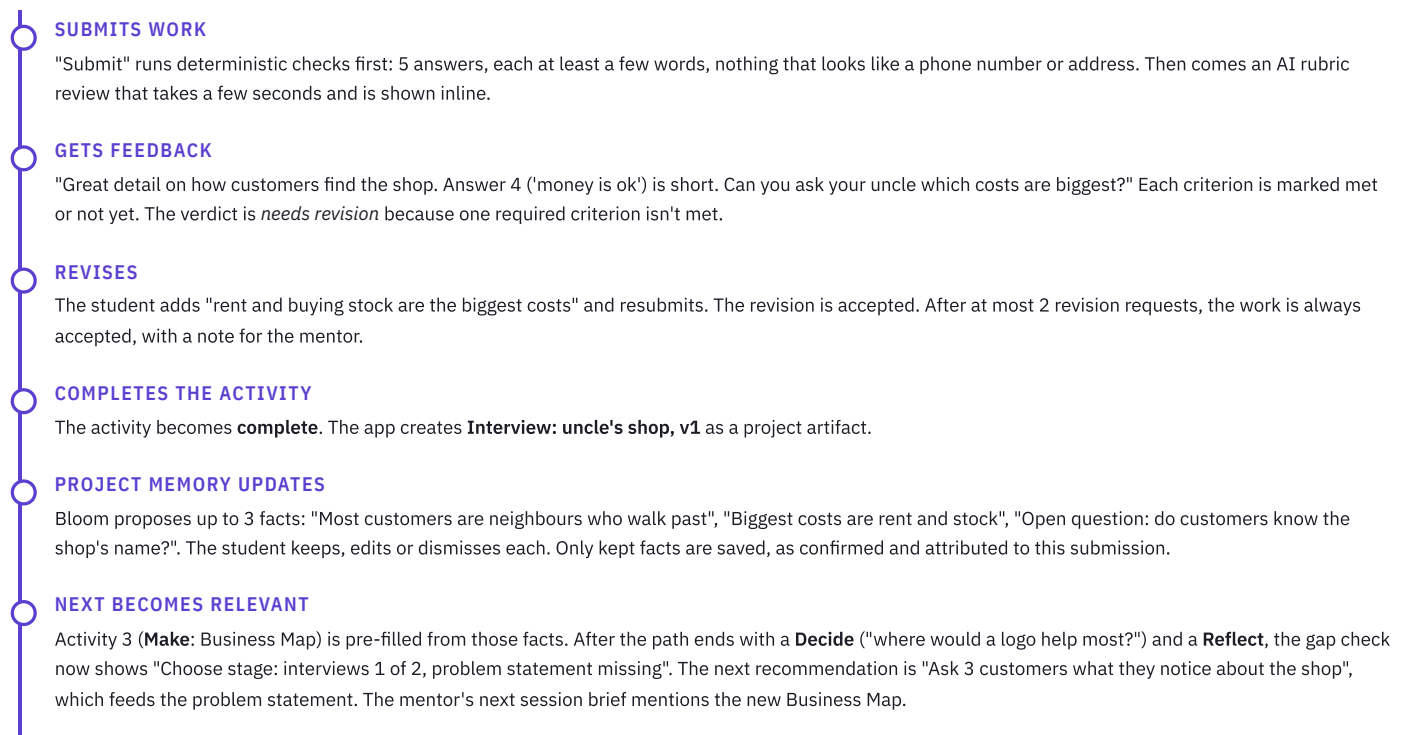
AI recommends and interprets. The application decides and stores the truth. Every meaningful path ends with something new on the student's project.

- 1. Keep the tables and evolve them.** `bloom_paths` stays the path. `bloom_tasks` becomes the activity table, gaining a type, a config and a richer state. The existing `kind` (learn, do, reflect) is kept as the path's three phases.
- 2. A short, fixed catalogue of 8 activity types**, each defined in code with its own evidence spec, completion rule and optional rubric. The AI planner can only assemble activities from this catalogue; it never invents new types.
- 3. Work is submitted, not ticked off.** Doing an activity creates a submission with evidence. Simple types complete instantly. Types that produce something get an immediate AI rubric review, which gives advice but caps revision requests at 2. Mentors review only at milestones.
- 4. Artifacts and project memory belong to the project, not the path.** An accepted submission can create a new artifact version. AI can propose project facts, but only a student or mentor confirmation writes them.
- 5. AI is a set of narrow, typed functions** (planner, coach, assessor, memory proposer, brief writer). They share one context assembler and one AI gateway that handles consent, limits and logging. It is not an autonomous agent: AI has no tools that write data.
- 6. The spine is the existing programme stages.** Each stage (Explore, Choose, Build, Present) lists the artifacts it expects. "What's next" is a deterministic gap check against that list, and the planner then fills in the detail.

01 Student experience, end to end

Example: an Explorer (age 13) in week 3 (Choose stage). Project area is business; working idea: "a logo and a brand for my uncle's shop".

- OPENS BLOOM**
The home page leads with **"Your project"**: a project card showing the current stage (Choose), what the stage expects (problem statement, at least 2 interviews), and what already exists (none yet). Below it is **"Recommended next"**: 1–3 paths picked because something is missing, for example "How a small business really works → produces a Business Map and an Interview". The curious-about field and "my own topic" stay as before.
- CHOOSSES A PATH**
The student taps "Start". The planner builds the path from the catalogue: 5 activities in 3 phases, aimed at *Business Map* as the target artifact. The path page shows that target at the top: "By the end you'll have: a money map of your uncle's shop."
- STARTS A STEP AND LEARNS**
Activity 1 is a **Learn** activity: 3 short cards on customers, what's sold and money in or out, with examples from a corner shop. There's an optional "I already know this, check me" link. A **Quick check** follows: sort 6 items ("rent", "selling sweets", "neighbours walking past", ...) into the 3 parts. 5/6 correct completes it. A wrong answer gets a one-line hint and one retry.
- TALKS TO BLOOM**
In any activity, "Ask Bloom" opens a thread for that activity. Bloom knows the project, the path, this activity and what the student has typed so far. Example: "What should I ask if my uncle is busy?" gets a short answer and a suggestion to pick the 3 most important questions.
- DOES A REAL TASK**
Activity 2 is an **Interview**. The student chooses 5 questions from suggestions or writes their own, and Bloom flags any closed (yes or no) ones. The student talks to their uncle offline, then records an answer under each question, saying who was interviewed by role ("my uncle, the shop owner"), never by name. They can add a photo of their handwritten notes.



02 Activity model

Eight types cover what Youth Idea Lab needs. Rank, sort and choose aren't separate types: they are variants of *Quick check*. Drawing, photo and file are evidence kinds, not activity types, so *Make* covers them. That keeps the catalogue small enough for the planner to use reliably and for the UI to build well.

TYPE	TEACHES	STUDENT DOES	DATA CAPTURED	COMPLETION	AI REVIEW	MENTOR REVIEW	WHEN
Learn	A concept, with an example from their own project	Reads 2–4 short cards; can test out	Opened and finished (viewed)	Reaching the last card, or passing the test-out check	No (content written by the planner)	No	MVP
Quick check choice · sort · rank	Checks understanding	Answers 3–6 items	Answers and score	Submitted. Below the threshold: a hint and 1 retry, then complete anyway	No. The answer key is deterministic and validated when the plan is made	No	MVP choice, sort · P2 rank
Respond	Explaining or applying an idea in their own words	Answers 1–3 prompts in text	Text per prompt	Accepted by review	Yes, against a rubric	No	MVP
Interview	Research with real people; asking open questions	Prepares questions, talks to a trusted adult or customer, records answers	Questions, answer per question, interviewee role, optional photo of notes	Accepted by review	Yes: completeness, specificity, open questions	Spot check, or at the milestone	MVP

TYPE	TEACHES	STUDENT DOES	DATA CAPTURED	COMPLETION	AI REVIEW	MENTOR REVIEW	WHEN
Make	Producing a project artifact	Fills a template (such as a business map) and/or uploads a photo or file (drawing, sketch, prototype)	Template fields as JSON; files	Accepted by review, which creates an artifact version	Yes: rubric per template; images via vision	At the milestone (see Mentor check)	MVP template, photo, file · LATER canvas
Try	Testing an idea: predict, run, compare	Writes a prediction, runs a small test, records the result	Prediction, method, result, what changed	Accepted by review	Yes	Optional	P2
Decide	Making and justifying a choice	Picks or ranks options and gives a reason that points to their own evidence	Choice, reason, linked artifacts or facts	Submitted (light AI nudge if the reason is empty)	Light, never blocking	Required if the decision is a milestone (such as the chosen problem)	MVP
Reflect	Thinking about their own learning	Answers 1–2 prompts	Text	Submitted with at least a minimal answer	Optional encouraging reply, never blocking	No (mentor can read it)	MVP
Mentor check	Human judgement at a milestone	Marks an artifact "ready for my mentor"	Reference to an artifact version; mentor verdict and comment	Mentor approves (in the app or during a live session)	No	Yes, this is the mentor review	MVP

Each type is defined once in code (for example `lib/bloom/activities/interview.ts`), holding its config schema (Zod), evidence spec, completion rule, rubric template and UI component. The planner outputs `{type, config}` pairs, which are validated against that schema before anything is saved.

03 Path structure

Decision: evolve, don't replace. Learn, Do, Reflect is already in the data (`kind`) and students understand it. It becomes the path's three *phases*, while the new *type* says what each activity actually is. Feedback and improvement aren't separate steps: they are the review loop inside each **Do** activity.

PHASE (EXISTING <code>KIND</code>)	SHOWN AS	ACTIVITY TYPES	RULE
learn	Understand	Learn, Quick check	1–2 activities, test-out allowed
do	Make	Respond, Interview, Make, Try	At least one activity that produces evidence. Each one runs submit, then feedback, then improve. Exactly one is the path's <i>core make</i> , which produces the target artifact.
reflect	Decide & reflect	Decide, Reflect, Mentor check	Ends the path. A Decide activity writes a project decision; a Reflect activity records learning.

Two path shapes:

- **Project path** (the default for recommendations): it has a `target_artifact_type`, and its core make activity creates or improves that artifact. 4–6 activities.
- **Curiosity path** (when the student's own topic has no project link): Understand, a short Respond or Try, then Reflect. It needs no artifact and keeps the open-ended curiosity Bloom has today. 3–4 activities.

The `depth` setting (quick, standard, deep) is replaced by the shape plus a size rule. A path the student marks "deep dive" may add a second Do activity.

04 Evidence model

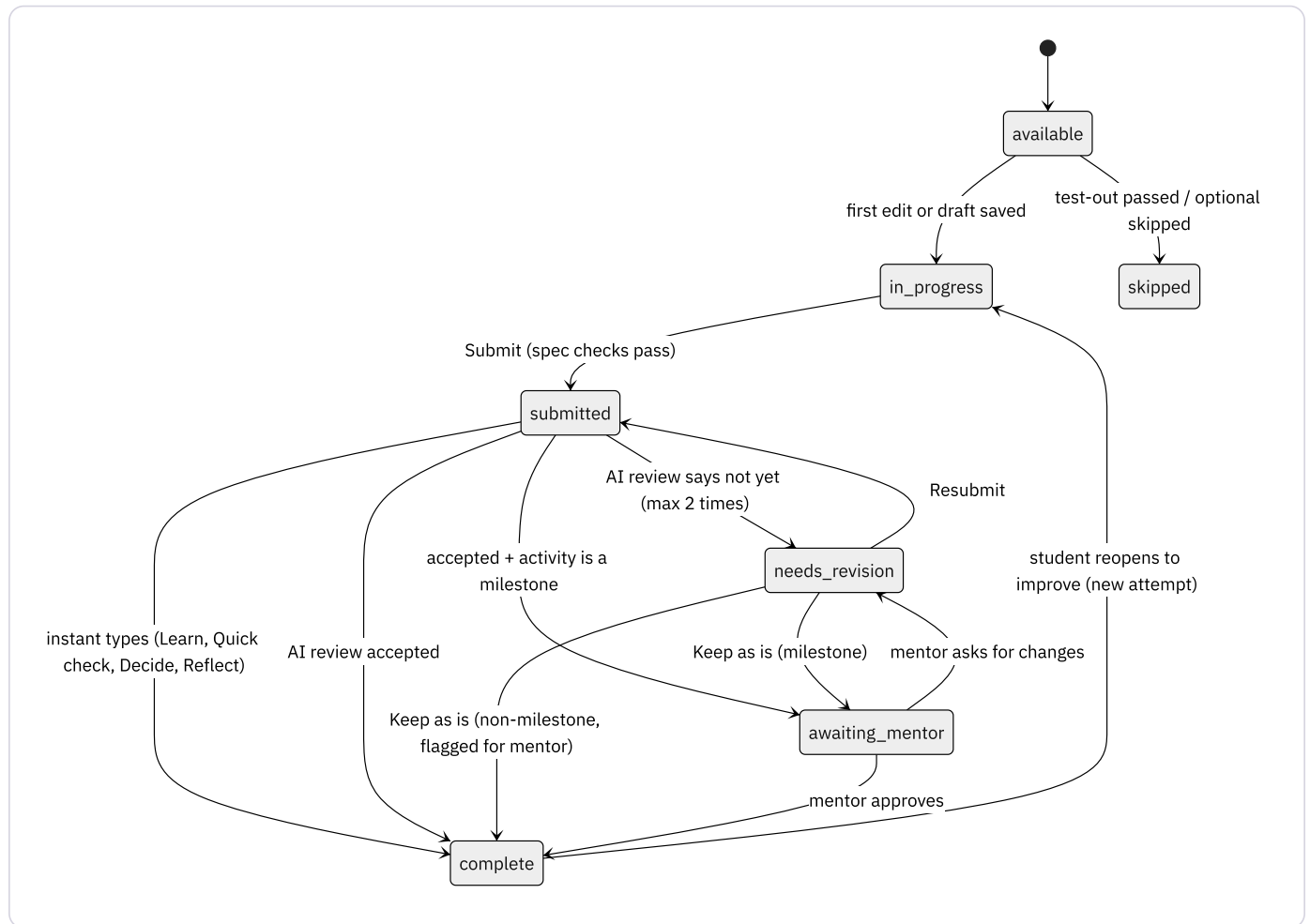
- **Evidence is what proves an activity was done**, and it is captured in a submission. An **artifact** is what the project keeps. An accepted submission *may* become an artifact version, but not all evidence does: a quiz answer never does.
- **Each activity type declares an evidence spec** made of slots, and the planner fills in the specifics (such as the 5 interview questions):

```
evidence: [  
  { slot: "answers", kind: "structured", schema: InterviewAnswers, required: true },  
  { slot: "notes", kind: "image", max: 3, required: false },  
  { slot: "interviewee_role", kind: "text", max: 60, required: true }  
]
```

EVIDENCE KIND	STORED AS	VALIDATION (APP)	WHEN
Text, short answer	<code>submission.content</code> JSON	Required, min and max length, PII patterns (phone numbers, emails, street addresses)	MVP
Selections, quiz answers, ranking	content JSON	Must match options in the activity config; scored against the key	MVP
Structured (template fields, interview Q&A)	content JSON validated by the template schema	Zod schema of the template version	MVP
Image (photo of a drawing, notes or prototype)	Storage bucket <code>bloom-evidence/{student}/{submission}/{...}</code> + file row	Type (jpeg, png, webp, heic converted), max 10 MB, max count; EXIF location removed on upload	MVP
File (pdf, slides)	Same bucket + file row	Allowlisted types, size	MVP
Link	content JSON	https only; no link shorteners	P2
Drawing (in-app canvas)	PNG in the bucket + optional stroke JSON	As image	LATER
Voice	Audio in the bucket + transcript	Length; transcript reviewed instead of the audio	LATER
Mentor confirmation	Assessment row (<code>by = mentor</code>)	Mentor of the student, or admin	MVP

- **Validation runs in four layers:** (1) the evidence spec, deterministic, which blocks submit; (2) safety: PII patterns and, later, image moderation, which blocks with an explanation or flags; (3) the AI rubric, which is advisory and can ask for a revision; (4) a mentor, only at milestones.
- **Honesty about offline work:** the app can't prove an interview happened. Evidence is good faith, made more credible by specific answers and the optional photo of notes. Mentors spot-check during sessions. Bloom never accuses a student of making work up.
- **Visibility:** same as Bloom today (`can_view_student`). Drafts are visible only to the student; submitted work is visible to the mentor and parent.

05 Completion and state machine



- **AI review is not a state the student waits in.** It runs inside the submit request (a few seconds, with a visible progress state) and is stored as an assessment. If the AI fails or times out, the work is **accepted** and flagged "not reviewed" for the mentor. AI problems never block a student.
- **Revision cap:** at most 2 AI-requested revisions per activity. After that the work is accepted with an "AI still had suggestions" note. "Keep as is" is always available, and it sends the work to the mentor queue without blocking.
- **Mentor review never blocks the rest of the path.** The student can carry on; only the milestone itself stays pending. A path completes when all *required* activities are complete or awaiting a mentor; optional activities don't count.
- **Path state** stays active, completed or archived, and is now recomputed in one place (a database function or service) after *every* activity change. That fixes the stale-status bugs found in the as-built analysis.
- **Mapping existing data:** todo becomes available, doing becomes in_progress, done becomes complete.

06 AI responsibilities

Decision: separate typed functions behind one gateway, not a single orchestrating agent. Each capability has its own prompt, input type, Zod output schema and tests. What they share:

- **Context assembler** (deterministic): builds the right slices of project memory and activity data for each capability, within a token budget.
- **AI gateway:** checks consent, limits per student, picks the model per capability, records every call in `ai_runs` (capability, model, tokens, latency, outcome, input hash), and turns failures into safe fallbacks. This replaces the scattered calls in today's `lib/ai.ts`.

A single agent with tools would make the app's state depend on model behaviour, which is exactly what this design avoids.

CAPABILITY	INPUT	OUTPUT	WHEN CALLED	ALLOWED	MUST NOT
Planner MVP	Project brief, stage and gap list, past paths (title, target, outcome), skills shown, student request, activity catalogue	Path: title, goal, target artifact type, summary, activities as {phase, type, title, config}	The student starts a recommended or own path	Choose types from the catalogue; write lesson cards, questions, quiz items with a key, template prompts, rubric weights within limits	Invent types; set state; skip validation; produce a plan without a Do activity for a project path
Coach (Ask Bloom, hints) MVP	Activity context, student's draft, last assessment, thread summary plus recent messages, project brief	Plain-text reply (≤ 150 words); optional suggested action chip ("add a step: ...")	The student asks; a quick check needs a hint	Explain, give examples, ask guiding questions, suggest an action for the student to confirm	Write the student's submission for them; change state; mark anything complete; reveal quiz keys
Assessor MVP	Activity config, rubric, submission content and images, relevant prior artifact	Per criterion: met or not yet, with a one-line reason; overall feedback text; suggested verdict	On submit, for Respond, Interview, Make and Try	Judge against the rubric; give specific, kind, actionable feedback	Grade with marks or percentages; compare the student with other students; set the state (the app maps the verdict); accuse the student of fabrication
Memory proposer MVP	Accepted submission, current confirmed facts	Up to 3 proposed facts {type, statement, source}, plus a summary of the new artifact version	After an accepted Interview, Make or Try	Propose insights, open questions and assumptions; summarise	Write facts directly; propose decisions (students make those in Decide); duplicate confirmed facts
Recommender P2	Gap list, facts, recent outcomes	1–3 ranked path ideas with reasons	On Bloom home (cached)	Rank and phrase ideas	Override the deterministic gap list. In the MVP this is done by the gap check plus the planner's suggestion mode.
Mentor brief writer P2	Changes since the last session: submissions, assessments, artifacts, facts, question topics, flags	5-bullet brief and 1–2 talking points per student	24 h before a session (sessions.starts_at), or on demand	Summarise and prioritise	Give parents this brief; judge character; include raw question text from flagged threads

All capabilities share the existing BLOOM_SYSTEM safety rules, extended with rules for evidence (no faces of other people, no addresses) and for assessment tone.

07 Application responsibilities

CONCERN	THE APPLICATION OWNS, DETERMINISTICALLY
Activity state	Every transition goes through one state service that applies the rules in section 5. AI verdicts are inputs, mapped by fixed rules.
Completion	Evidence spec satisfied, rules per type, revision cap, path recomputation. Progress is weighted by required activities.
Evidence ownership	<code>student_id</code> is copied by trigger (as today); storage paths are prefixed with the student; signed URLs only.
Permissions	RLS via <code>can_view_student</code> and <code>mentors_student</code> (as today); column grants; mentor actions through security-definer RPCs.
Project records	Facts are written only on a confirm action by a student or mentor. Project fields are edited only by people.
Mentor approval	Only a mentor of the student, or an admin, can move an activity out of <code>awaiting_mentor</code> .
Auditability	The existing <code>audit_log</code> trigger, plus <code>ai_runs</code> , plus assessments kept as their own rows (never overwritten).
Artifact versions	Versions are immutable once created; "current" and "final" are pointers changed by people; a new version always links its source submission.
Plans	Planner output is schema-validated, then normalised (counts, lengths, key checks). Path and activities are inserted in one transaction (RPC), which fixes the non-atomic insert.
Limits and consent	<code>bloom_ai</code> consent checked in the gateway; daily AI-call budget per student; safety flags routed to the mentor and admin.

08 Project memory

Project memory is a set of **records**, not one AI-written document: the project row, the project's facts, its artifacts' current versions and, later, skill evidence. A short **project brief** for prompts is compiled from those records by code, so it is always reproducible.

INFORMATION	SHAPE	WHERE	WHO WRITES
Original idea, area, status	Structured	<code>projects</code> (existing)	Student (project page)
Problem being solved	Free text, plus the Problem Statement artifact once made	<code>projects.problem</code> → artifact	Student
Target user or customer	Short text field	<code>projects.target_user</code> (new)	Student; can be suggested via a fact
Research findings	Fact type <code>insight</code> , free-text statement	<code>project_facts</code>	AI proposes, then the student confirms
Decisions	Fact type <code>decision</code> , with a reason	<code>project_facts</code>	Student, through a Decide activity
Assumptions and open questions	Fact types <code>assumption</code> and <code>question</code>	<code>project_facts</code>	AI proposes or the student adds; resolved by later facts

INFORMATION	SHAPE	WHERE	WHO WRITES
Artifacts	Structured, versioned	<code>artifacts</code> , <code>artifact_versions</code>	From accepted submissions
Mentor feedback	Free text	Assessments (by mentor), <code>mentor_note</code> , course <code>feedback</code>	Mentor
Student reflections	Free text	Reflect submissions	Student
Demonstrated skills	Structured: skill key, level, source	<code>skill_evidence</code> P2	App rule: accepted activity tagged with a skill
Current and next milestone	Derived, not stored	Stage from the week plus the gap check	App

- **Safe updates.** AI output lands as `status = proposed` and is shown to the student as "Bloom noticed... Save to my project?" with edit, keep and dismiss. Only keeping it (or a mentor confirming) sets `confirmed` . A changed fact is *superseded* by a new row, never edited in place, so earlier states stay traceable. Mentors can add, edit (by superseding) or retire facts.
- **Brief budget.** The compiled brief holds the project fields, up to 12 confirmed facts (decisions first, then newest), and a one-line summary of each current artifact version. Its cap is about 1,200 tokens. It is cached per project and rebuilt when any input changes.

09 Project artifacts

ARTIFACT TYPE	PRODUCED BY	TEMPLATE OR FORM	TYPICAL STAGE	WHEN
Interview set	Interview	Questions, answers per question, interviewee role, notes photos	Explore, Choose	MVP
Business map	Make (template)	Customers · offer · how customers find you · money in · money out · helpers	Explore, Choose	MVP
Problem statement	Make (template), then Mentor check	"[Who] struggles with [what] because [why]", plus evidence links	Choose (milestone)	MVP
Customer profile	Make (template)	Who, needs, habits, quotes from interviews	Choose	P2
Concept sketch (logo, brand, product)	Make (photo or file)	Image(s) plus "what it shows" and "why"	Build	MVP (generic image artifact)
Prototype	Make (photo, file or link)	Media plus what works and what doesn't	Build (milestone)	P2
Experiment results	Try	Prediction, test, result, change	Build	P2
Pitch	Make (file or link), then Mentor check	Slides or a script, plus a rehearsal note	Present (milestone)	P2

- **Drafts and versions:** an in-progress submission is the draft. Each accepted submission creates an immutable `artifact_version` (v1, v2 and so on) linked to its submission. `artifacts.current_version_id` points to the newest accepted version, and `final_version_id` is set by the student or mentor for the portfolio.
- **Feedback** attaches to versions through assessments (AI or mentor), so the history shows how the work improved.
- **Portfolio:** final versions of a project, shown on the existing public portfolio when `projects.is_public` and the `public_portfolio` consent allow. Images go through review before becoming public. **P2**
- **Course submissions:** programme activities can also produce artifact versions later, so the course and Bloom feed one project record. **P2**

10 Adaptive learning

- **The spine is a milestone map in code**, keyed by stage. It lists the artifacts each stage expects: Explore (an interest or exploration note, 1 interview), Choose (2+ interviews, a problem statement checked by the mentor), Build (a concept, a prototype checked by the mentor, 1 experiment), Present (a pitch checked by the mentor, a portfolio). The **gap check** compares it with the project's artifacts. That deterministic list drives "Recommended next".
- **No repetition**: if the target artifact already has a current version, the planner is told to make an "improve" path (v2), not a new "create" path. The planner also receives the titles, targets and outcomes of the last 12 paths.
- **Skip what's known**: Learn activities offer a test-out quick check. Passing it marks Learn as skipped and records skill evidence (P2).
- **Micro-lessons when needed**: wrong quick-check answers, or a "not yet" on a concept criterion, trigger an inline Coach explanation inside the activity. It isn't a new activity, so the path doesn't grow.
- **Reference earlier work**: activity configs can reference artifacts and facts by id (for example, the Business Map is pre-filled from interview facts), and the brief includes artifact summaries.
- **Gaps across paths** **P2** : criteria the student repeatedly misses (such as "answers lack specifics") feed the planner as "practise: ...".

11 Ask Bloom

There is one **thread per activity** plus one general thread per path, with messages kept as rows. The existing `bloom_questions` become messages in these threads. Context is assembled by scope with fixed budgets, so prompts don't grow with use:

CONTEXT SLICE	INCLUDED	BUDGET (TOKENS, APPROX.)
System and safety rules	Always	600
Project brief	Always	≤1,200
Path: goal, target, activity titles and states	Always	≤400
Current activity: config and the student's current draft or last submission	In an activity thread	≤1,500
Latest assessment feedback for this activity	If one exists	≤300
Latest mentor note or comment on this path or artifact	If one exists	≤200
Thread: rolling summary plus the last 6 messages	Always	≤1,800

- **Growth control**: scoped threads, truncation by recency, and a rolling summary rebuilt every 8 messages by a cheap call **P2** . The MVP uses the last 8 messages only. Search across threads (vector retrieval) is **LATER** and only if needed.
- **Limits**: the Coach can suggest an action ("Add a step: ask two customers") that appears as a button. Only the student's click acts on it. It can't complete work or reveal answer keys.
- **Safety**: wellbeing or safety topics get the existing "talk to a trusted adult" handling plus a mentor and admin flag **MVP** .

12 Mentor role

- **Checkpoints at milestones only**: problem statement (Choose), prototype (Build) and pitch (Present) **MVP** for the problem statement, **P2** for the rest. Decisions marked as milestones, such as "my chosen problem", are also checked.
- **The mentor queue** (one list per group, sorted by urgency): awaiting mentor; "kept as is" and "not reviewed" items; students stuck (2 revisions in a row, or no activity for 5+ days, reusing `loadLastActive`); safety flags.
- **Per-student view**: the project brief, the artifact timeline with versions, paths and progress, recent assessments, and the existing timeline.
- **Mentor actions**: approve or ask for changes (with a comment), confirm, edit or retire a project fact, and the existing per-path note.
- **Mentor brief** **P2** : before each live session, a per-student card covering what they made since last time, where they got stuck, the topics they asked Bloom about, open milestones, and 1–2 suggested talking points. Every point links to its source.

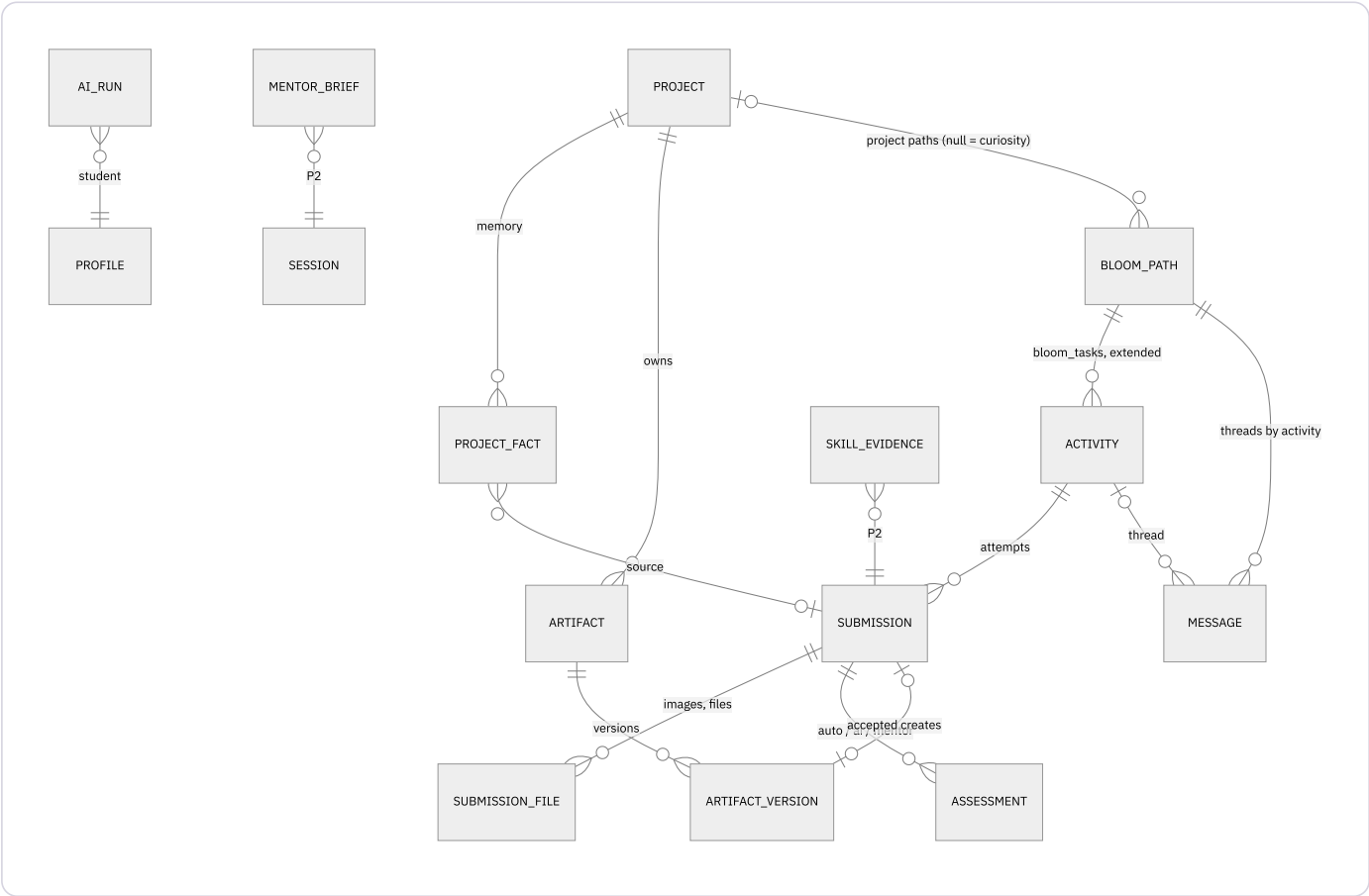
13 Parent role

PARENTS SEE	PARENTS DON'T SEE (BY DEFAULT)
Path titles, progress and what each path produced; current and final artifact versions; mentor checkpoint outcomes; weekly progress cards (existing); consent controls; data export (existing)	Drafts; the back-and-forth of AI assessments; proposed (unconfirmed) facts; mentor briefs

Ask Bloom transcripts are an **open decision**. Today's consent text promises parents can "read every question and answer". Options: (a) keep full visibility, the current promise; (b) show topics only, with full text for safety-flagged threads. Either way, safety flags go to the mentor and admin first and follow the safeguarding policy. Students should be told plainly what parents can see.

14 Conceptual data model

Only entities that earn their place. A separate conversation table isn't needed, because a thread is identified by (path, activity). Evidence items are submission content plus a file table; there is no generic evidence table.



ENTITY	ESSENCE	WHEN
Project (existing projects)	+ target_user ; the anchor for memory and artifacts	MVP
BloomPath (existing)	+ project_id , shape , target_artifact_type , plan_version	MVP
Activity (existing bloom_tasks)	+ type , config JSON, required , is_milestone , origin (ai or student), richer status , revision_count	MVP
Submission	Attempt n of an activity: content JSON, status, submitted_at	MVP
SubmissionFile	Storage path, type, size, moderation state	MVP
Assessment	By auto, ai or mentor; criteria results; verdict; feedback; ai_run_id	MVP

ENTITY	ESSENCE	WHEN
Artifact / ArtifactVersion	Type, title, current and final pointers / immutable content, summary, source submission	MVP
ProjectFact	Type, statement, status (proposed, confirmed, superseded, dismissed), source, confirmed_by	MVP
Message (evolves bloom_questions)	path_id, activity_id?, role (student or bloom), text, flags	MVP
AiRun	Capability, model, tokens, latency, outcome, input hash	MVP
SkillEvidence	Skill key (catalogue in code), level, source submission	P2
MentorBrief	Session, student, content, generated_at	P2

Activity types, templates, rubrics, the milestone map and the skill catalogue live **in code**, versioned with the app, not in the database. A version number is stored with each activity so old configs stay readable.

15 Migration strategy

ACTION	WHAT
KEEP	bloom_paths , bloom_tasks (as activities), the RLS pattern (can_view_student), triggers (copy student, completion stamps, audit), bloom_ai consent, mentor_note and its RPC, export_student_data , BloomPathDetail slots, timeline, mentor group totals, loadStudentOverview , course submissions as they are
EXTEND	bloom_tasks : type, config, required, is_milestone, origin, revision_count; the status enum gains available , in_progress , submitted , needs_revision , awaiting_mentor , complete and skipped (renamed from todo, doing, done in one migration). bloom_paths : project_id, shape, target_artifact_type, plan_version. projects : target_user. lib/ai.ts becomes the gateway plus capability modules. The export function adds the new tables.
MIGRATE	Existing tasks become type = legacy , which completes on click (today's behaviour), so old paths keep working with no backfill of evidence. A non-empty reflection becomes a submission with that text. bloom_questions become message pairs (student, then bloom). kind stays as the phase.
DEPRECATE	bloom_tasks.reflection (read-only, then dropped after migration); depth (kept for legacy display); bloom_paths.ai_generated (replaced by per-activity origin); bloom_questions (replaced by a view, then dropped)
NEW	bloom_submissions , bloom_submission_files , bloom_assessments , artifacts , artifact_versions , project_facts , bloom_messages , ai_runs , the storage bucket bloom-evidence , the activity catalogue and players in code, the state service, the context assembler and the mentor queue

The as-built defects are fixed along the way: the wiped-note bug disappears with the submission model; path and activities are inserted in one transaction; path status is recomputed in one place; adding steps recomputes status.

16 MVP vs later

MVP / PHASE 1

- Activity types: Learn, Quick check (choice, sort), Respond, Interview, Make (template, photo, file), Decide, Reflect, Mentor check
- Submissions, evidence spec checks, image and file upload with EXIF stripping, PII patterns
- AI Assessor with rubrics, revision loop (cap 2), fail-open behaviour
- New state machine and transactional path creation
- Artifacts and versions: interview set, business map, problem statement, generic image or file
- Project facts with propose and confirm; the compiled brief
- Planner using the catalogue, the brief and the gap check (milestone map)
- Ask Bloom threads per activity, with the last 8 messages
- Mentor queue plus one checkpoint (problem statement)
- AI gateway: consent, daily limit, ai_runs log
- Legacy path support

PHASE 2

- Try activity type; rank variant; link evidence
- Mentor brief before sessions
- Skill evidence and the skill view
- Test-out on Learn; cross-path gap patterns
- Recommender ranking; rolling thread summaries
- Customer profile, prototype, experiment and pitch artifacts; prototype and pitch checkpoints
- Parent artifact view; portfolio from final versions
- Course submissions feeding artifacts

LATER

- Drawing canvas, voice evidence with transcription
- Image moderation service; peer review
- Urdu and other languages for prompts and rubrics
- Vector retrieval across threads; spaced review
- Analytics on learning outcomes across cohorts

17 Worked example: "How a Small Business Really Works"

Path shape: project path. Target artifact: **Business map**. It also produces an **Interview set** and a **decision**.

1 · Understand: The 3 big parts of any small shop

| Today: a Learn task with a text explanation, then Mark done.

TYPE	Learn (3 cards: customers, what's sold, money in and out) + Quick check (sort)
INTERACTION	Reads the cards, then sorts 6 cards ("rent", "sweets", "neighbours walking past", "buying stock", "school children", "selling phone credit") into the 3 parts
EVIDENCE	Sort answers (required)
AI FEEDBACK	None for grading (the key is deterministic). A one-line Coach hint on a wrong item.
COMPLETION	Instant on submit. Below 5/6: a hint and 1 retry, then complete anyway.
PROJECT UPDATE	None (skill evidence "business basics" in P2)

2 · Make: Ask your uncle 5 simple questions

Today: a Do task with instructions; Mark done records nothing.

TYPE	Interview
INTERACTION	Picks or edits 5 questions (Bloom flags closed ones), interviews the uncle offline, records an answer per question, sets interviewee role "uncle, shop owner", optional photo of notes
EVIDENCE	Required: 5 answers, interviewee role. Optional: up to 3 photos.
AI FEEDBACK	Rubric: all questions answered; answers are specific (numbers, examples); at least 3 open questions; money question answered. Needs revision if a required criterion isn't met.
COMPLETION	Accepted by review (max 2 revisions, then accepted with a note)
PROJECT UPDATE	Artifact <i>Interview set: uncle's shop v1</i> . Proposed facts: customer insight, cost insight, open question, which the student confirms.

3 · Make: Draw your shop money map (core make)

Today: a Do task to draw on paper; nothing is captured.

TYPE	Make, template <code>business_map</code> + optional photo of the hand drawing
INTERACTION	Fills the six map boxes, pre-filled with suggestions from the confirmed interview facts, which the student edits. Can upload a photo of a paper version instead of typing every box (at least 3 boxes typed).
EVIDENCE	Required: customers, offer, money in, money out. Optional: how customers find you, helpers, photo.
AI FEEDBACK	Rubric: money in and out both present; consistent with the interview; customers described specifically. The photo is read with vision for consistency only, never graded on drawing skill.
COMPLETION	Accepted by review
PROJECT UPDATE	Artifact <i>Business map v1</i> , the path's target. A revision later creates v2.

4 · Decide: Spot where a logo would help customers

Today: a Do task; the conclusion lives only in the student's head.

TYPE	Decide (rank variant), linked to the project idea (logo)
INTERACTION	From 5 "moments customers notice the shop" (sign, bags, receipts, WhatsApp status, shop front), chooses the top 2 and writes why, pointing to the map or the interview. Optional photo of the current sign, with the guidance "no people, no street name".
EVIDENCE	Required: 2 choices and a reason. Optional: photo.
AI FEEDBACK	Light: a nudge if the reason doesn't mention any evidence. Never blocks.
COMPLETION	Instant on submit
PROJECT UPDATE	Decision fact (student-authored, confirmed): "A logo matters most on the shop sign and carrier bags, because most customers are walk-ins". Sets up the Build-stage concept sketch.

5 · Reflect: What did I learn about the shop?

Today: a Reflect task whose answer is optional and can be wiped.

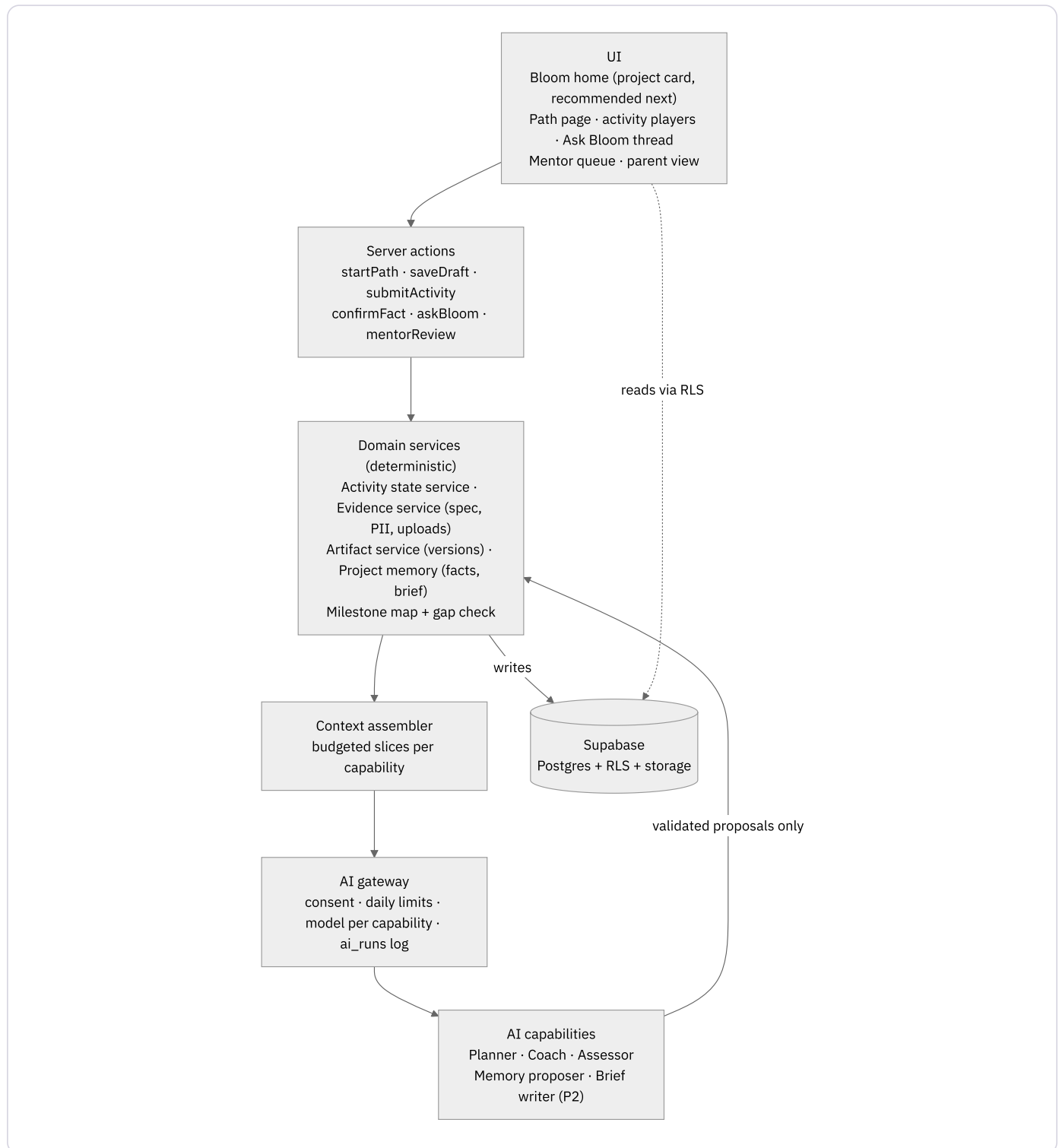
TYPE	Reflect + choose next
INTERACTION	Two prompts: "What surprised you?" and "What will you do next for your project?". Then picks a next path from the 2–3 shown, which come from the gap check.
EVIDENCE	Required: 1 prompt answered (minimal length)
AI FEEDBACK	Optional one-line encouraging reply
COMPLETION	Instant. Completes the path.

Path outcome recorded (artifacts made, decision). The gap check moves to "Choose: interviews 1/2, problem statement missing". The mentor's next brief lists the new business map.

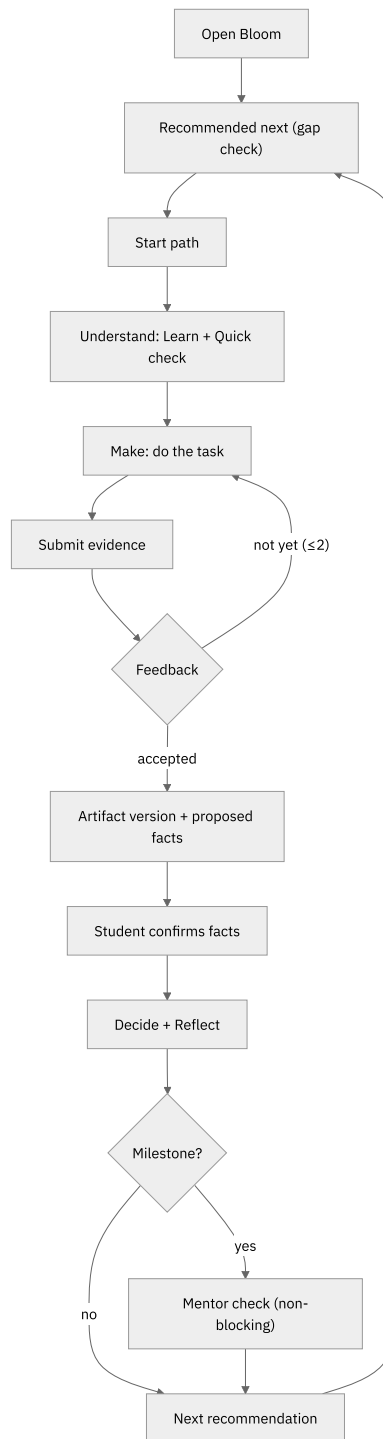
Mentor involvement: none required for this path, because the business map isn't a milestone. The mentor sees it in their queue only if the student chose "Keep as is" or the AI review failed. The milestone, the problem statement, comes in a later path.

18 Final architecture

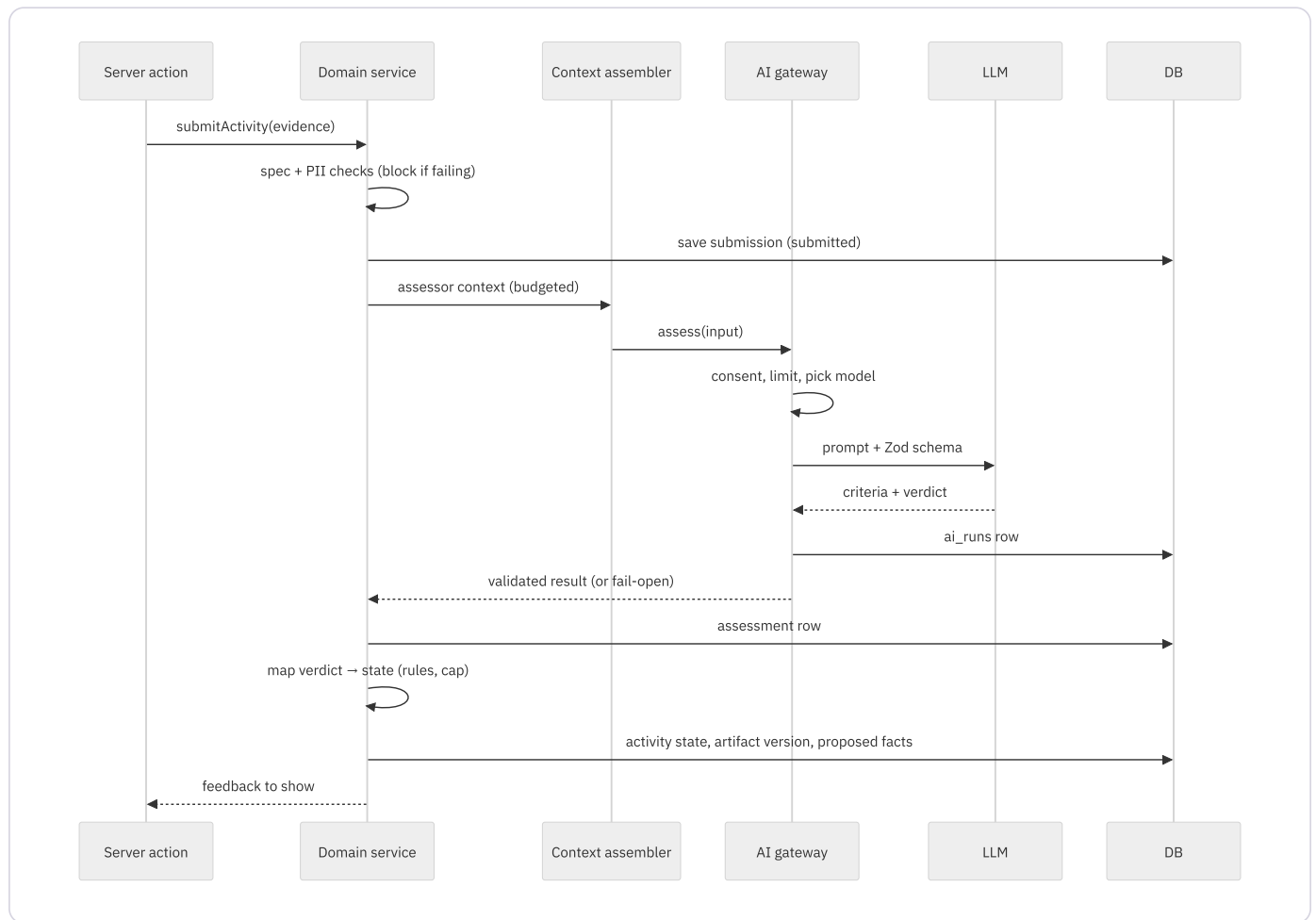
A. Target architecture



B. Student flow



C. AI orchestration (one call)



D. Conceptual data model

See section 14.

E. Activity state machine

See section 5.

F. MVP scope

See section 16. In one sentence: 8 activity types (Try excluded), submissions with photo and file evidence, rubric-based AI feedback with a capped revision loop, versioned artifacts for 3 templates plus generic images, confirmable project facts, a planner driven by a milestone gap check, threaded Ask Bloom, and a mentor queue with one milestone checkpoint.

G. Existing components to reuse

- Tables `bloom_paths` and `bloom_tasks`, triggers, RLS helpers (`can_view_student`, `mentors_student`, `is_admin`), audit log, consent model, export function
- `loadStudentOverview`, `currentWeek`, `stageForWeek`, the `stages` table (milestone map keys), `sessions` (brief timing)
- `BloomPathDetail` slot pattern, `BloomPathCard`, `Timeline`, `LastActive`, `loadLastActive`, `loadBloomTotals`, `ProgressBar`, `SubmitButton`
- The `BL00M_SYSTEM` safety rules, the `structured()` Zod pattern, `betaZodOutputFormat`
- Course storage patterns (`submission_files`, `signFiles`) as the template for evidence uploads
- The mentor note RPC pattern for mentor review

H. Components and data that need modification

- `lib/ai.ts` → gateway plus capability modules; `bloomContext` → context assembler
- `actions.ts`: `createPath` (transactional, catalogue-based), `updateTask` (→ state service), `askQuestion` (→ threads), `addTask` (student activities choose a type)

- `bloom_tasks` and `bloom_paths` columns and the status enum; `projects.target_user`
- `TaskControls` → activity players; `AskForm` → thread; the Bloom home gets the project card and recommendations
- Mentor student page (queue, artifacts, facts); parent child page (artifacts)
- The `bloom_ai` consent text: photos and project work are now sent to the AI, which needs re-consent

I. New components and data

- Tables: submissions, submission files, assessments, artifacts, artifact versions, project facts, messages, AI runs; storage bucket `bloom-evidence`
- Code: activity catalogue (types, configs, evidence specs, rubrics, templates), state service, evidence service (uploads, EXIF, PII), artifact service, memory service and brief compiler, milestone map and gap check, AI gateway
- UI: 8 activity players, the fact confirmation card, the artifact view with versions, the mentor queue

J. Major risks and open product decisions

ITEM	WHY IT MATTERS	NEEDS
Quality and tone of AI assessment for 12–18 year-olds	Harsh or wrong feedback discourages; overly soft feedback teaches nothing	Rubric review by educators; sample-based evaluation before launch; the cap and fail-open behaviour
Cost and latency per submission	Every Make or Interview submission is an AI call, some with images	A model per capability; daily limits; measure via <code>ai_runs</code>
Photos of children's surroundings	Faces, addresses and shop names can identify family members	EXIF stripping, upload guidance, mentor-only visibility for flagged images, a moderation plan, a retention policy
Consent scope change	Today's consent covers age group, step and project, not photos or work	New consent version and re-consent flow
Parent visibility of Ask Bloom	Autonomy versus the current promise	Product and safeguarding decision (section 13)
Mentor capacity	Queues can overwhelm mentors with large groups	Milestones only; a target number of checkpoints per student per stage
Course activities versus Bloom	Two places where students submit work	Decide whether the course activities later run on the same activity engine (P2 decision)
Offline work honesty	An interview can't be verified	Accept good faith; mentor spot checks; no accusations by AI
Language	Students may prefer Urdu; the rubrics are English	Decide the MVP language policy
Low bandwidth and mobile	Photo uploads on mobile data	Client-side image compression; drafts saved locally
Template authoring effort	Each template needs a schema, rubric, UI and examples	Start with 3; add more from usage